this with the `IHttpClientFactory` features you've already seen by extracting the body of the `GetRates()` function from listing 33.4 into a separate service. But `IHttpClientFactory` has deeper support for this pattern.

`IHttpClientFactory` supports typed clients. A *typed client* is a class that accepts a configured `HttpClient` in its constructor. It uses the `HttpClient` to interact with the remote API and exposes a clean interface for consumers to call. All the logic for interacting with the remote API is encapsulated in the typed client, such as which URL paths to call, which HTTP verbs to use, and the types of responses the API returns. This encapsulation makes it easier to call the third-party API from multiple places in your app by using the typed client.

The following listing shows an example typed client for the exchange rates API shown in previous listings. It accepts an `HttpClient` in its constructor and exposes a `GetLatestRates()` method that encapsulates the logic for interacting with the third-party API.

Listing 33.5 Creating a typed client for the exchange rates API

```
public class ExchangeRatesClient
{
    private readonly HttpClient _client;                    ❶
    public ExchangeRatesClient(HttpClient client)           ❶
    {
        _client = client;
    }

    public async Task<string> GetLatestRates()              ❷
    {
        var response = await _client.GetAsync("latest");    ❸
        response.EnsureSuccessStatusCode();                 ❸

        return await response.Content.ReadAsStringAsync();  ❸
```